

BX GitHub Repositories

Handbuch für Shopbetreiber

Ziel dieser Anleitung

Dieses Handbuch beschreibt die Einrichtung des Moduls BX GitHub Repositories für den Produktivbetrieb.

Nach Abschluss der Schritte kann der Shop:

- auf die eigenen GitHub-Repositories zugreifen,
 - Release-ZIP-Dateien abrufen,
 - ZIP-Dateien im Shop-Ordner **download** speichern,
 - und nur die für den jeweiligen Shop freigegebenen Repositories verwenden.
-

Ergebnis am Ende der Einrichtung

Folgende drei Werte müssen erfolgreich eingerichtet sein:

1. GitHub App ID
2. Installation ID
3. Private Key (PEM)

Zusatz:

- Verbindungstest im Modul ist erfolgreich.
 - Repositories der Installation können geladen und ausgewählt werden.
-

Voraussetzungen

1. GitHub-Account mit Rechten zum Anlegen und Installieren einer GitHub App.
 2. Admin-Zugriff auf den Shop.
 3. Schreibrechte auf den Shop-Ordner **download**.
 4. Zugriff auf die Repositories, die später synchronisiert werden sollen.
-

Schritt 1: GitHub App anlegen

1. In GitHub oben rechts auf das Profil klicken.
2. **Settings** öffnen.
3. **Developer settings**, nach unten scrollen und links im Menu öffnen.
4. **GitHub Apps** auswählen.
5. **New GitHub App** klicken.
6. Einen eindeutigen Namen vergeben, z. B. **shopname-release-sync**.
7. **Homepage URL** eintragen (z. B. Shop-URL).
8. **Callback URL** nur setzen, wenn Benutzer-Login über GitHub benötigt wird. Für uns nicht relevant.

9. Wenn keine Webhooks benötigt werden: Webhooks deaktivieren. Für reine Versions-Synchronisation sind Webhooks nicht erforderlich.
10. Berechtigungen minimal setzen:
 - Repository permission **Metadata: Read-only**
 - Repository permission **Contents: Read-only**
11. Unter Installation festlegen, wo die App installierbar ist.
12. App speichern.

Hinweis: So wenig Rechte wie möglich vergeben (Least Privilege).

Schritt 2: GitHub App installieren

1. In der GitHub App auf **Install App** gehen.
2. Ziel-Account oder Ziel-Organisation auswählen.
3. Repository-Zugriff auf **Selected repositories** setzen.
4. Nur die Repositories auswählen, die der Shop verwenden soll.
5. Installation bestätigen.

Wichtig: Diese Auswahl bestimmt, welche Repositories der Shop technisch überhaupt sehen darf.

Schritt 3: App ID ermitteln

1. In den GitHub-App-Einstellungen die **App ID** ablesen.
 2. Wert notieren.
-

Schritt 4: Installation ID ermitteln

1. In die Installationsansicht der App wechseln.
2. Die Installations-ID aus der URL oder Installationsansicht entnehmen.
3. Wert notieren.

Beispiel: Bei einer URL mit `.../settings/installations/12345678` ist **12345678** die Installation ID.

Schritt 5: Private Key (PEM) erzeugen

1. In der GitHub-App-Konfiguration den Bereich **Private keys** öffnen.
2. **Generate a private key** klicken.
3. Die heruntergeladene PEM-Datei sicher speichern.
4. Die PEM-Datei für die Modulkonfiguration bereithalten.

Wichtig:

- Private Keys nicht per E-Mail versenden.
- Private Key niemals in Tickets oder Chats posten.
- Bei Verdacht auf Leak sofort neuen Key erzeugen und alten widerrufen.

Schritt 6: Daten im Shop-Modul eintragen

1. Im Shop-Admin das Modul BX GitHub Repositories öffnen.
 2. Folgende Felder befüllen:
 - **GitHub App ID**
 - **Installation ID**
 - **Private Key (PEM)** Datei hochladen
 3. **Verbindung testen** ausführen.
 4. Bei Erfolg **Einstellungen speichern**.
-

Schritt 7: Template-Produkt anlegen und konfigurieren

Vor dem ersten Anlegen von Download-Produkten muss ein Template-Produkt im Shop vorhanden sein.

1. Im Shop-Admin ein neues Produkt in der gewünschten Ziel-Kategorie anlegen.
2. Dieses Produkt bewusst als Template pflegen, also nicht als regulär verkaufbares Produkt verwenden.
3. Das Template-Produkt auf **inaktiv** setzen und dauerhaft inaktiv lassen.
4. Die **Sortorder** des Template-Produkts auf **9999** setzen.
5. Alle Standardwerte pflegen, die später für automatisch angelegte Produkte übernommen werden sollen.
6. Dem Template-Produkt in den Artikelmerkmalen **Downloads** einen Download-Eintrag zuordnen.
7. Als Downloaddatei für dieses Template bewusst die Datei **dummy.zip** hinterlegen.
8. Besonders wichtig sind außerdem Kategoriezuordnung, Steuerklasse und weitere gewünschte Produkt-Grundeinstellungen.
9. Die **products_id** dieses Template-Produkts notieren.
10. Diese ID in den Moduleinstellungen als **Template-Produkt ID** hinterlegen.

Wichtig:

- Das Template-Produkt ist nur eine Vorlage und bleibt immer inaktiv.
 - Neu angelegte Repository-Produkte werden aus diesem Template kopiert.
 - Das erste Download-Attribut des Template-Produkts wird mitkopiert.
 - Die hinterlegte Datei **dummy.zip** wird beim angelegten Repository-Produkt automatisch durch den Namen der jeweiligen Repository-ZIP ersetzt.
 - Das Template bestimmt insbesondere die Kategoriezuordnung und viele weitere Standardparameter der automatisch angelegten Produkte.
-

Schritt 8: Repository-Liste laden und Auswahl festlegen

1. Nach erfolgreichem Verbindungstest die Repository-Liste laden.
2. Die gewünschten Repositories auswählen und Auswahl speichern.
3. Repositories herunterladen

Importziel: Das ZIP-Dateien werden im Shop-Ordner **download** gespeichert.

Wichtig: Jedes verwendete Repository muss mindestens einen Tag mit Versionsnummer haben (z. B. **v2.0.5**). Nur dann kann das Modul eine ZIP-Datei laden. Ohne Version-Tag ist kein ZIP-Download möglich.

Beispiel:

```
git tag -a v2.0.5 -m "BX Products Video v2.0.5"  
git push origin v2.0.5
```

Typische Fehlerbilder und Lösungen

Fehler 401 / 403 beim Verbindungstest

Mögliche Ursache:

- App ID, Installation ID und PEM gehören nicht zusammen.
- App ist nicht für das Ziel-Repository installiert.
- Berechtigungen sind zu knapp.

Lösung:

1. Werte erneut prüfen.
2. Installation prüfen.
3. Permissions für den Anwendungsfall prüfen.

Repository-Liste ist leer

Mögliche Ursache:

- Installation hat keine Repositories freigegeben.
- Falscher Account oder falsche Organisation.

Lösung:

1. In GitHub die Installation aufrufen.
2. **Selected repositories** kontrollieren.
3. Gewünschte Repositories freigeben.

ZIP-Import funktioniert nicht

Mögliche Ursache:

- Release-Asset passt nicht zum Muster.
- Dateirechte im Ordner **download** fehlen.

Lösung:

1. Asset-Name und Pattern prüfen.
2. Schreibrechte auf **download** prüfen.
3. Testlauf erneut starten.

Sicherheits-Checkliste vor Go-Live

1. Nur minimale Permissions aktiv.
 2. Installation auf **Selected repositories** gesetzt.
 3. Keine sensiblen Daten im Klartext in Logs.
 4. Private Key geschützt gespeichert.
 5. Testlauf erfolgreich.
 6. Download-Datei korrekt im Ordner **download**.
 7. Geplante Task-Ausführung aktiviert.
-

Wartung und Rotation

1. Private Key regelmäßig rotieren.
2. Nicht mehr benötigte Repositories aus der Installation entfernen.
3. Nach Rechteänderungen Verbindungstest erneut ausführen.
4. Fehlerprotokolle regelmäßig kontrollieren.